

TD03 : Requêtes préparées et association de classes

Table des matières

1) Problématique.....	1
2) Exercices	1
3) Conclusion.....	7

1) Problématique

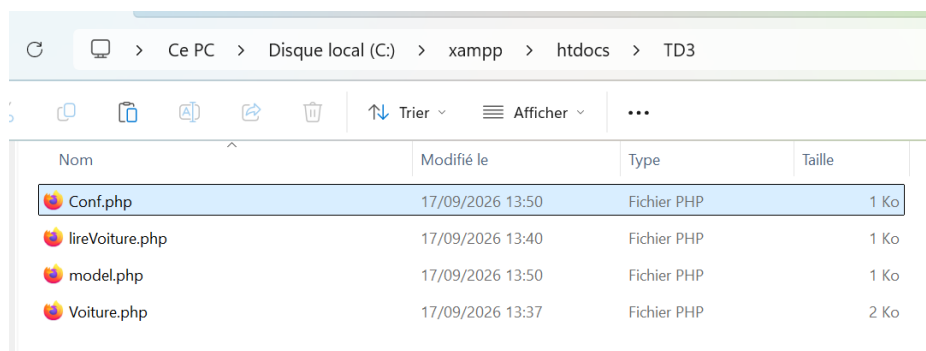
Ce TD est le prolongement direct du TD2 sur l'enregistrement des données en base avec PDO. Nous souhaitons sécuriser nos requêtes contre les injections SQL à l'aide des requêtes préparées (prepare et execute). Ensuite, nous allons modéliser et programmer les associations entre plusieurs tables pour concevoir la base d'un site de covoiturage : associer un conducteur à un trajet, relier plusieurs passagers grâce à une table intermédiaire, et effectuer des requêtes de sélection avec jointures SQL (INNER JOIN). Pour réaliser ce TP, nous utilisons XAMPP (Apache, MySQL, PhpMyAdmin) et Notepad++.

2) Exercices

Exercice 1 : Les requêtes préparées

1) Copie des fichiers dans le dossier TD3

On commence par créer un nouveau dossier nommé TD3 dans le répertoire htdocs de XAMPP. Ensuite, on y copie-colle les fichiers créés lors du TD précédent : Conf.php, Model.php, Voiture.php et lireVoiture.php.



Nom	Modifié le	Type	Taille
Conf.php	17/09/2026 13:50	Fichier PHP	1 Ko
lireVoiture.php	17/09/2026 13:40	Fichier PHP	1 Ko
model.php	17/09/2026 13:50	Fichier PHP	1 Ko
Voiture.php	17/09/2026 13:37	Fichier PHP	2 Ko

2) Déclaration de la fonction getVoitureByImmat dans Voiture.php

Pour éviter les failles d'injection SQL causées par une concaténation directe, on utilise une requête préparée avec PDO. On associe un marqueur nommé (:nom_tag) à la valeur de l'immatriculation, puis on prépare et exécute la requête :

```
public static function getVoitureByImmat($immat) {
    $sql = "SELECT * from voiture WHERE immatriculation = :nom_tag";
    $req_prep = Model::$pdo->prepare($sql);
    $values = array(
        "nom_tag" => $immat
    );
    $req_prep->execute($values);
    $req_prep->setFetchMode(PDO::FETCH_CLASS, 'Voiture');
    $stab_voit = $req_prep->fetchAll();

    if (empty($stab_voit)) {
        return false;
    }
    return $stab_voit[0];
}
```

3) Test de la fonction dans lireVoiture.php

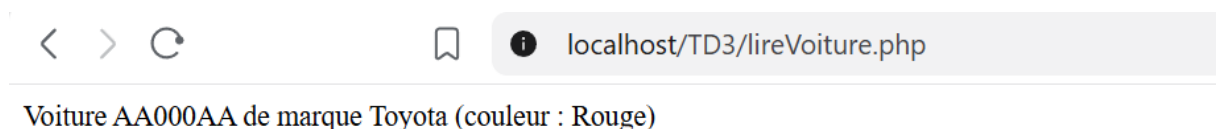
Dans lireVoiture.php, on teste notre nouvelle méthode statique en cherchant une voiture existante avec son immatriculation :

```
<?php
require_once "Model.php";
require_once "Voiture.php";

$voitureTrouvee = Voiture::getVoitureByImmat('AA000AA');

if ($voitureTrouvee) {
    $voitureTrouvee->afficher();
} else {
    echo "Aucune voiture trouvée avec cette immatriculation.";
}
?>
```

On ouvre ensuite le navigateur à l'adresse <http://localhost/TD3/lireVoiture.php> pour voir le résultat :



(Le résultat est le même qu'au TD2 mais désormais l'injection SQL n'est plus possible car on ne récupère pas toute la table mais simplement l'immatriculation indiqué dans lireVoiture.php)

Exercice 2 : Insertion d'un objet en BDD avec save()

1) Création de la fonction save() dans Voiture.php

On ajoute une méthode save() dans la classe Voiture. Comme c'est une méthode d'instance, elle accède aux attributs de l'objet courant grâce à \$this. On utilise une requête INSERT INTO préparée avec les tags correspondants. Attention : pas besoin de faire de fetch() car INSERT INTO ne retourne aucun résultat :

```
public function save() {
    $sql = "INSERT INTO voiture (immatriculation, marque, couleur) VALUES (:immat, :marque, :couleur)";
    $req_prep = Model::$pdo->prepare($sql);
    $values = array(
        "immat" => $this->immatriculation,
        "marque" => $this->marque,
        "couleur" => $this->couleur
    );
    $req_prep->execute($values);
}
```

2) Test de la fonction dans lireVoiture.php

On teste l'enregistrement en créant une nouvelle instance de Voiture puis en appelant save() :

```
<?php
require_once "Model.php";
require_once "Voiture.php";

$v = new Voiture("Renault", "Bleu", "BB111BB");
$v->save();

echo "Voiture enregistrée avec succès !<br>";
$v->afficher();
```

Résultat :

✓ Affichage des lignes 0 - 1 (total de 2, traitement en 0,0003 seconde(s).)

SELECT * FROM `voiture`

☐ Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table

Options supplémentaires

immatriculation	marque	couleur
AA000AA	Toyota	Rouge
BB111BB	Renault	Bleu

Exercice 3 : Liaison avec le formulaire de création de voiture

Je ne fais pas l'exercice 3 car il demande d'avoir fait TD1 que l'on a déjà fait il y a longtemps mais dont je n'ai plus les fichiers. Ce sera aussi le cas pour l'exercice 7, 8 et 9 (même s'ils sont bonus).

Exercice 4 : Création des tables de covoiturage (utilisateur et trajet)

1 et 2) Table utilisateur et insertion des données

Dans PhpMyAdmin, on crée la table utilisateur avec ces spécifications :

- login : VARCHAR(32), clé primaire
- nom : VARCHAR(32)
- prenom : VARCHAR(32)

Ensuite on insère quelques utilisateurs de test (ex: 'dupont', 'leclerc', 'durand') /

✓ Affichage des lignes 0 - 2 (total de 3, traitement en 0,0003 seconde(s).)

```
SELECT * FROM `utilisateur`
```

☐ Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Options supplémentaires

			login	nom	prenom
☐	Éditer	Copier	Supprimer	dupontj	Dupont Jean
☐	Éditer	Copier	Supprimer	durandm	Durand Marie
☐	Éditer	Copier	Supprimer	martinluc	Martin Luc

3 et 4) Table trajet et insertion des trajets

On crée ensuite la table trajet avec les colonnes suivantes :

- id : INT, clé primaire avec AUTO_INCREMENT (case A_I cochée)
- depart : VARCHAR(32)
- arrivee : VARCHAR(32)
- date : DATE
- nbplaces : INT
- prix : INT
- conducteur_login : VARCHAR(32)

On insère ensuite des trajets en associant des logins d'utilisateurs existants au champ conducteur_login :

✓ Affichage des lignes 0 - 2 (total de 3, traitement en 0,0003 seconde(s).)

```
SELECT * FROM `trajet`
```

☐ Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Options supplémentaires

				id	depart	arrivee	date	nbplaces	prix	conducteur_login
☐	Éditer	Copier	Supprimer	1	Montpellier	Paris	2026-10-15	3	45	dupontj
☐	Éditer	Copier	Supprimer	2	Toulouse	Bordeaux	2026-10-18	2	20	durandm
☐	Éditer	Copier	Supprimer	3	Lyon	Marseille	2026-10-20	4	30	dupontj

Exercice 5 : Association conducteur (clé étrangère)

Réponses aux questions :

- Comment implémenter l'association conducteur tenant compte de sa multiplicité ?

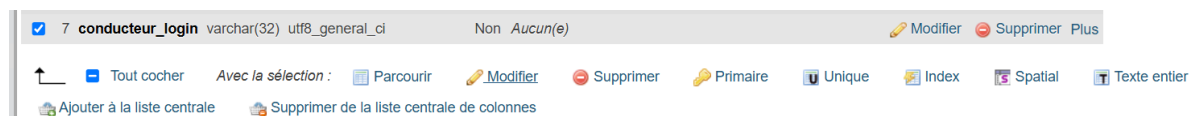
=> Comme un trajet possède exactement un seul conducteur, on ajoute simplement le champ conducteur_login dans la table trajet.

- Quelle fonctionnalité des BDD garantit la correspondance à tout moment ?

=> Il s'agit des clés étrangères (Foreign Keys (FK)).

1) Déclaration de l'index sur conducteur_login

Dans l'onglet Structure de la table trajet sur PhpMyAdmin, on clique sur l'action 'Index' en face de conducteur_login.

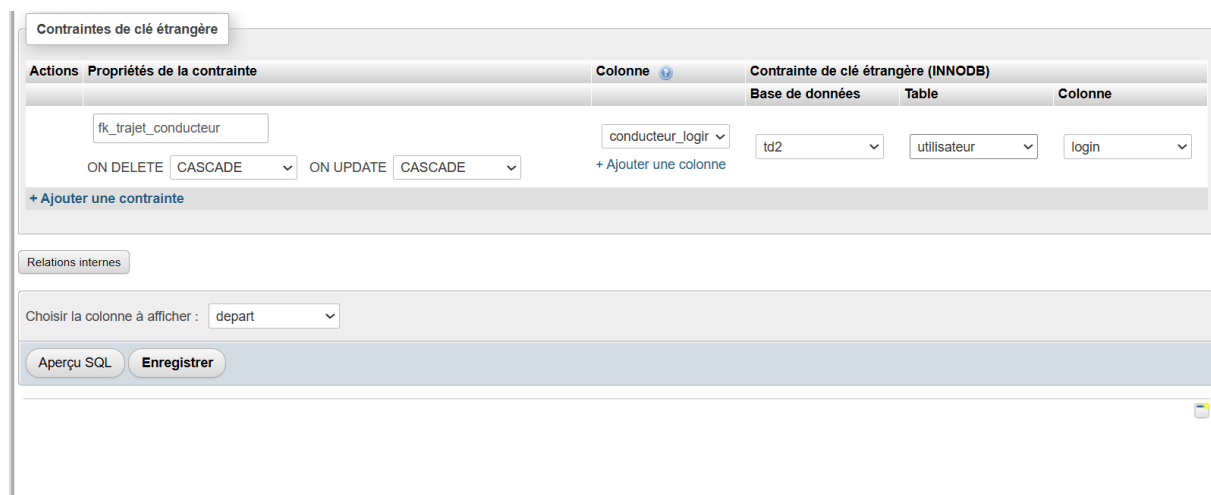


Résultat :



2) Configuration de la clé étrangère en CASCADE

Toujours dans la table trajet, on va dans 'Vue relationnelle' et on crée la contrainte de clé étrangère reliant trajet.conducteur_login vers utilisateur.login, en sélectionnant ON DELETE CASCADE et ON UPDATE CASCADE :



Puis on enregistre.

Exercice 6 : Association passagers (table de liaison)

Réponses aux questions :

- Comment implémenter l'association passager tenant compte des multiplicités ?

L'association étant de plusieurs à plusieurs (0..n côté trajet et 0..n côté utilisateur), on doit créer une table d'association (nommée passager).

- Quelle est la clé primaire de la table passager ?

C'est le couple (trajet_id, utilisateur_login).

1, 2 et 3) Création de la table passager et clés étrangères

On crée la table passager avec deux colonnes : trajet_id (INT) et utilisateur_login (VARCHAR 32). On sélectionne les deux champs ensemble pour former la clé primaire composite. Ensuite dans la vue relationnelle, on lie trajet_id à trajet.id et utilisateur_login à utilisateur.login avec CASCADE :

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
☐	1 trajet_id	int(11)			Non	Aucun(e)			Modifier Supprimer Plus
☐	2 utilisateur_login	varchar(32)	utf8_general_ci		Non	Aucun(e)			Modifier Supprimer Plus

4 et 5) Insertion d'associations et test de ON DELETE CASCADE

On inscrit des passagers dans la table passager. Ensuite, on supprime un utilisateur dans la table utilisateur. On constate que toutes les lignes de la table passager associées à cet utilisateur ont été automatiquement supprimées en cascade.

	trajet_id	utilisateur_login
☐ Éditer Copier Supprimer	1	durandm
☐ Éditer Copier Supprimer	1	martinluc
☐ Éditer Copier Supprimer	2	dupontj
☐ Éditer Copier Supprimer	4	durandm

Exercice 7 : Affichage des passagers avec une jointure SQL

1) Fonction findPassagers(\$id) dans Trajet.php

On crée la méthode statique findPassagers(\$id) dans Trajet.php en utilisant une jointure INNER JOIN entre la table utilisateur et la table passager :

```
public static function findPassagers($id) {
    $sql = "SELECT u.* FROM utilisateur u
            INNER JOIN passager p ON u.login = p.utilisateur_login
            WHERE p.trajet_id = :id_tag";
    $req_prep = Model::$pdo->prepare($sql);
    $values = array("id_tag" => $id);
    $req_prep->execute($values);
    $req_prep->setFetchMode(PDO::FETCH_CLASS, 'Utilisateur');
    return $req_prep->fetchAll();
}
```

2 et 3) Pages testFindUtil.php et formulaire formFindUtil.php

On crée un formulaire formFindUtil.php (méthode GET) qui demande l'identifiant d'un trajet et envoie la requête à testFindUtil.php. La page testFindUtil.php récupère le trajet_id et affiche la liste des passagers inscrits :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Recherche de passagers</title>
</head>
<body>
  <form method="get" action="testFindUtil.php">
    <label for="id_trajet">Identifiant du trajet :</label>
    <input type="number" name="trajet_id" id="id_trajet" required />
    <input type="submit" value="Rechercher" />
  </form>
</body>
</html>
```

Passagers inscrits au trajet 1 :

Utilisateur : Marie Durand (durandm)

Utilisateur : Luc Martin (martinluc)

3) Conclusion

Ce TD nous a permis de comprendre l'importance capitale des requêtes préparées pour bloquer les attaques par injection SQL, ainsi que la gestion des liaisons entre tables au niveau de la base (clés étrangères, ON DELETE CASCADE) et au niveau du code PHP avec des jointures SQL (INNER JOIN). Grâce à l'association des classes en POO, notre application de covoiturage manipule désormais des objets cohérents et sécurisés en lien direct avec MySQL.